

Lesson 4: Language Models & Generation

You've learned retrieval - now let's explore the generation side of RAG. This lesson covers how LLMs work and how to integrate retrieved context effectively.

What is a Large Language Model?

A **Large Language Model (LLM)** is a neural network trained to: - Understand natural language - Generate human-like text - Follow instructions - Complete tasks (summarization, Q&A, translation, etc.)

Popular LLMs: - OpenAI: GPT-4, GPT-3.5 - Anthropic: Claude 3 (Opus, Sonnet, Haiku) - Google: Gemini, PaLM - Meta: Llama 2, Llama 3 - Mistral: Mistral 7B, Mixtral

How LLMs Work (Simplified)

1. Training

LLMs are trained on massive text datasets:

Training Data:

- Books, articles, websites (trillions of words)
- Code repositories (GitHub, etc.)
- Conversations, Q&A pairs

Training Process:

1. Show the model: "The cat sat on the ____"
2. Model predicts: "mat" (or "chair", "floor", etc.)
3. Compare prediction with actual next word
4. Adjust model to improve predictions
5. Repeat billions of times

Result: A model that understands patterns in language and can generate coherent text.

2. Inference (Using the Model)

When you use an LLM:

Input (Prompt):

"Explain quantum computing in simple terms."

Model Process:

1. Converts text to tokens (numbers)
2. Processes tokens through neural network
3. Generates probabilities for next token
4. Samples a token (usually highest probability)
5. Repeats until complete response

Output:

"Quantum computing uses quantum mechanics to perform..."

3. Key Concepts

Tokens: Text chunks (roughly 4 characters or $\frac{3}{4}$ words)

"Hello world" = ["Hello", " world"] = 2 tokens

"ChatGPT is amazing" = ["Chat", "G", "PT", " is", " amazing"] = 5 tokens

Context Window: Maximum input + output length - GPT-3.5: 16K tokens (~12,000 words) - GPT-4: 128K tokens (~96,000 words) - Claude 3: 200K tokens (~150,000 words)

Temperature: Controls randomness (0 = deterministic, 1 = creative)

RAG Integration: The Generation Step

In RAG, the LLM generates responses using retrieved context:

The RAG Prompt Template

```
prompt_template = """
Use the following context to answer the question.
If you cannot answer based on the context, say so.

Context:
{retrieved_documents}

Question: {user_question}

Answer:
"""
```

Example: RAG in Action

```
# User asks
question = "What is the capital of France?"

# System retrieves (from Lesson 3)
retrieved_docs = [
    "Paris is the capital and largest city of France.",
    "The Eiffel Tower is located in Paris, France.",
    "France is a country in Western Europe."
]

# Create augmented prompt
prompt = f"""
Use the following context to answer the question.
If you cannot answer based on the context, say so.

Context:
1. Paris is the capital and largest city of France.
2. The Eiffel Tower is located in Paris, France.
3. France is a country in Western Europe.

Question: What is the capital of France?

Answer:
"""

# LLM generates
response = llm.generate(prompt)
# Output: "Based on the context, Paris is the capital of France."
```

Effective Prompting for RAG

1. Basic Prompt Structure

```
system_prompt = """
You are a helpful assistant that answers questions based on provided context.
Always cite which context piece you used to answer.
If the context doesn't contain the answer, say so clearly.
"""

user_prompt = """
Context:
{context}

Question: {question}

Answer:
"""
```

2. Instructions for Better Responses

Add constraints:

```
prompt = """
Answer the question based on the context below.

Rules:
- Use only information from the context
- Cite the document number you're referencing
- If uncertain, say "I don't know based on the provided context"
- Keep answers concise and factual

Context:
[1] {doc1}
[2] {doc2}
[3] {doc3}

Question: {question}

Answer:
"""
```

3. Few-Shot Examples

Show the model how to respond:

```
prompt = """
Answer questions based on the provided context.

Example 1:
Context: Python was created by Guido van Rossum in 1991.
Question: When was Python created?
Answer: According to the context, Python was created in 1991.

Example 2:
Context: The Earth orbits the Sun.
Question: What is the population of Mars?
Answer: I cannot answer this question based on the provided context.

Now answer this:
Context: {context}
Question: {question}
Answer:
"""
```

4. Chain-of-Thought Prompting

Encourage step-by-step reasoning:

```
prompt = """
Context: {context}

Question: {question}

Let's think step by step:
1. What information from the context is relevant?
2. How does this information answer the question?
3. What is the final answer?

Answer:
"""
```

Choosing an LLM for RAG

Factors to Consider

Factor	Consideration
Quality	GPT-4, Claude 3 Opus (best but expensive)
Speed	GPT-3.5, Claude 3 Haiku (fast, cheaper)
Cost	Open-source models (free but need hosting)
Context Length	Claude 3 (200K), GPT-4 (128K) for long docs
Privacy	Self-hosted models for sensitive data

Cost Comparison (Approximate)

Model	Input (per 1M tokens)	Output (per 1M tokens)
GPT-4 Turbo	\$10	\$30
GPT-3.5 Turbo	\$0.50	\$1.50
Claude 3 Opus	\$15	\$75
Claude 3 Sonnet	\$3	\$15
Claude 3 Haiku	\$0.25	\$1.25
Llama 3 (hosted)	Free	Free (hosting costs)

Recommendations

- For learning:** GPT-3.5 Turbo (cheap, good quality)
- For production:** Claude 3 Sonnet or GPT-4 Turbo
- For high volume:** GPT-3.5 or self-hosted Llama 3
- For sensitive data:** Self-hosted models (Llama, Mistral)

Using LLMs with Python

OpenAI API

```
from openai import OpenAI

client = OpenAI(api_key="your-api-key")

def generate_response(prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.7,
        max_tokens=500
    )
    return response.choices[0].message.content

# Usage
prompt = "Explain RAG in one sentence."
answer = generate_response(prompt)
print(answer)
```

Anthropic API (Claude)

```
from anthropic import Anthropic

client = Anthropic(api_key="your-api-key")

def generate_response(prompt: str) -> str:
    message = client.messages.create(
        model="claude-3-sonnet-20240229",
        max_tokens=500,
        messages=[
            {"role": "user", "content": prompt}
        ]
    )
    return message.content[0].text

# Usage
prompt = "Explain RAG in one sentence."
answer = generate_response(prompt)
print(answer)
```

Open-Source Models (with Ollama)

```

import requests

def generate_response(prompt: str) -> str:
    response = requests.post(
        "http://localhost:11434/api/generate",
        json={
            "model": "llama3",
            "prompt": prompt,
            "stream": False
        }
    )
    return response.json()['response']

# Usage (requires Ollama running locally)
prompt = "Explain RAG in one sentence."
answer = generate_response(prompt)
print(answer)

```

Complete RAG Generation Example

```

from openai import OpenAI
import chromadb

class RAGSystem:
    def __init__(self, openai_api_key: str):
        # Initialize LLM
        self.llm = OpenAI(api_key=openai_api_key)

        # Initialize vector DB
        self.chroma_client = chromadb.Client()
        self.collection = self.chroma_client.create_collection("docs")

    def add_documents(self, documents: list[str]):
        """Add documents to vector database"""
        ids = [f"doc_{i}" for i in range(len(documents))]
        self.collection.add(documents=documents, ids=ids)

    def retrieve(self, query: str, n_results: int = 3) -> list[str]:
        """Retrieve relevant documents"""
        results = self.collection.query(
            query_texts=[query],
            n_results=n_results
        )
        return results['documents'][0]

    def generate(self, query: str, context: list[str]) -> str:
        """Generate response using LLM"""
        # Format context
        context_str = "\n".join([f"[{i+1}] {doc}" for i, doc in enumerate(context)])

        # Create prompt
        prompt = f"""
Answer the question based on the context below.
Cite the document numbers you reference.
If you cannot answer based on the context, say so.

```



```

Context:
{context_str}

Question: {query}

Answer:
"""

    # Generate response
    response = self.llm.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.7
    )

    return response.choices[0].message.content

def query(self, question: str) -> dict:
    """Complete RAG pipeline"""
    # Step 1: Retrieve
    relevant_docs = self.retrieve(question)

    # Step 2: Generate
    answer = self.generate(question, relevant_docs)

    return {
        "question": question,
        "answer": answer,
        "sources": relevant_docs
    }

# Usage
rag = RAGSystem(openai_api_key="your-key")

# Add documents
rag.add_documents([
    "Paris is the capital of France.",
    "The Eiffel Tower is in Paris.",
    "Python is a programming language.",
])

# Query
result = rag.query("What is the capital of France?")
print(f"Q: {result['question']}")
print(f"A: {result['answer']}")
print(f"\nSources:")
for i, source in enumerate(result['sources'], 1):
    print(f"    [{i}] {source}")

```

Output:

Q: What is the capital of France?
A: According to document [1], Paris is the capital of France.

Sources:

- [1] Paris is the capital of France.
- [2] The Eiffel Tower is in Paris.
- [3] Python is a programming language.

Handling Common Issues

Issue 1: Model Ignores Context

Problem: LLM uses its training data instead of provided context

Solution: Be explicit in prompt

```
prompt = """
IMPORTANT: Answer ONLY using the context below.
Do NOT use external knowledge.

Context: {context}
Question: {question}
"""
```

Issue 2: Hallucinations

Problem: LLM makes up information

Solutions: - Lower temperature (e.g., 0.3 instead of 0.7) - Add explicit constraints - Request source citations - Post-process: verify claims against sources

```
# Force citation
prompt = """
Answer the question and cite the exact text from context.

Context: {context}
Question: {question}
"""
```

```
Format:
Answer: [your answer]
Citation: "[exact quote from context]"
"""
```

Issue 3: Too Verbose

Problem: LLM generates long responses

Solution: Constrain length

```
prompt = """
Answer in 1-2 sentences maximum.

Context: {context}
Question: {question}
Answer:
"""

# Or use max_tokens parameter
response = llm.generate(prompt, max_tokens=50)
```

Issue 4: Context Too Long

Problem: Retrieved docs exceed context window

Solutions: 1. Retrieve fewer documents

```
results = retrieve(query, n_results=3) # Instead of 10
```

1. Summarize context first

```
def summarize_docs(docs):
    summary_prompt = f"Summarize these documents:\n{docs}"
    return llm.generate(summary_prompt)
```

```
context = summarize_docs(retrieved_docs)
```

1. Use model with larger context

```
# Use GPT-4 (128K) or Claude 3 (200K) instead of GPT-3.5 (16K)
```

Advanced Generation Techniques

1. Streaming Responses

Show responses as they're generated:

```
def stream_response(prompt: str):
    stream = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        stream=True
    )
    for chunk in stream:
        if chunk.choices[0].delta.content:
            print(chunk.choices[0].delta.content, end="")
```

2. Multi-Turn Conversations

Maintain conversation history:

```
class ConversationalRAG:
    def __init__(self):
        self.history = []

    def chat(self, user_message: str):
        # Retrieve context
        context = self.retrieve(user_message)

        # Add to history
```

```

        self.history.append({
            "role": "user",
            "content": f"Context: {context}\n\nQuestion: {user_message}"
        })

    # Generate with history
    response = llm.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=self.history
    )

    # Add response to history
    self.history.append({
        "role": "assistant",
        "content": response.choices[0].message.content
    })

    return response.choices[0].message.content

```

3. Self-Reflection

Ask model to verify its answer:

```

# First pass: Generate answer
answer = generate(query, context)

# Second pass: Verify
verification_prompt = f"""
Original question: {query}
Context: {context}
Proposed answer: {answer}

Is this answer:
1. Supported by the context?
2. Accurate?
3. Complete?

If not, provide a corrected answer.
"""

verified_answer = generate(verification_prompt, context)

```

What You've Learned

- How LLMs generate text and key concepts
- How to structure prompts for RAG effectively

- How to choose the right LLM for your use case
- Complete RAG generation pipeline with code
- Common issues and how to solve them

Practice Exercises

1. **Experiment with prompts:** Try different prompt structures and see how they affect responses
2. **Compare models:** Use GPT-3.5 and Claude with the same RAG system
3. **Test edge cases:** What happens when context doesn't contain the answer?
4. **Build conversational RAG:** Implement multi-turn conversations with context

Next Steps

In [Lesson 5: Building a Simple RAG System](#), you'll: - Build a complete RAG system from scratch - Process real documents - Create a command-line interface - Test and evaluate your system

Further Reading

- [OpenAI Prompt Engineering Guide](#)
- [Anthropic Prompt Engineering Guide](#)
- [LangChain Documentation](#)

- [Attention Is All You Need \(Transformer Paper\)](#)

[← Lesson 3: Vector Databases & Retrieval](#) | [Next: Building a Simple RAG System →](#)